Optimizing Placements for Teaching Assistants using Intelligent Multi-agent Analytics (OPTIMA)

Team Number:

500-ZW-4-TASch

Current Project Manager:

Md Samiul Haque, mdsamiul.haque@ucalgary.ca (30169845)

Team Members:

Elias Poitras-Whitecalf (30193066)

Erioluwa Olubadejo (30187041)

Adol Awan (30237232)

Academic Advisor:

Leanne Dawson

Sponsoring Company:

Schulich Studio

Sponsor Representatives:

Emily Marasco, eamarasc@ucalgary.ca

Tariq Al Shoura, tariq.alshoura@ucalgary.ca

Table of contents

Section	Page
3. Glossary	4
4. Short Bios	9
5. Project Motivation and Objectives	12
6. Project Scope and Deliverables	14
7. Preliminary Design of the Proposed Solution	20
8. Alternative Solutions/Methodologies	34
9. IP and Other Legal Agreements	39
10. Technical Specifications	40
11. Materials, Supplies, Tools and Cost Estimates	43
12. Project Delivery Risks and Mitigation Strategies	47
13. Major Technical Tasks and Milestones	50
14. Roles and Responsibilities	54
15. Measures of Success and Validation Tests	58
16. Team Logbook	61

17. Appendix 62

18. References 70

AI USE STATEMENT:

Generative AI tools were used during the preparation of this document: [ChatGPT (model/version)] for locating and summarizing sources, and [Claude (model/version)] for drafting, editing, and reviewing sections against the course rubric. No TA applicant data, sponsor data, or other personal information was entered into any AI tool. All AI-assisted content, including every cited source, was verified by team members against the original sources, and final content was reviewed and approved by all four members.

3. Glossary

Term / Abbreviation	Definition
AI (Artificial Intelligence)	Artificial intelligence (AI) is technology that enables computers and machines to simulate human learning, comprehension, problem solving, decision making, creativity and autonomy. [1]
AI Agent	An artificial intelligence (AI) agent is a system that autonomously performs tasks by designing workflows with available tools [2], such as analyzing TA applications, ranking candidates, or generating schedules.
Agentic AI	Agentic AI is an artificial intelligence system that can accomplish a specific goal with limited supervision. It consists of AI agents—machine learning models that mimic human decision-making to solve problems in real time. In a multiagent system, each agent performs a specific subtask required to reach the goal and their efforts are coordinated through AI orchestration. [3]

Multi-Agent System (MAS)	A multi-agent system (MAS) consists of multiple artificial intelligence (AI) agents working collectively to perform tasks on behalf of a user or another system. [4]
LLM (Large Language Model)	Large language models (LLMs) are a category of deep learning models trained on immense amounts of data, making them capable of understanding and generating natural language and other types of content to perform a wide range of tasks. LLMs are built on a type of neural network architecture called a transformer which excels at handling sequences of words and capturing patterns in text. [5]
OpenAI Agents SDK	A software development kit used to build agentic AI apps in a lightweight, easy-to-use package with very few abstractions. [6]
TA (Teaching Assistant)	A student or individual responsible for supporting instructors with teaching-related activities, such as tutorials, laboratories, and grading. [7]
Candidate Suggestion Ranking	The process of ordering TA applicants based on their qualifications and predefined evaluation criteria. [8]
TA Scheduling	The process of suggesting potential teaching assistants to available courses, sections, or time slots while considering scheduling constraints. [9]
Keyword Extraction	The process of identifying relevant skills, qualifications, and experiences from unstructured TA application responses in a

	way that mitigates bias based on language (masculine tones, etc.) [10]
RAG (Retrieval-Augmented Generation)	Retrieval Augmented Generation (RAG) connects LLMs to external data sources so they can ground their responses in up-to-date, domain-specific information. Instead of relying only on training data, an RAG system retrieves relevant documents at query time and feeds them to the model alongside the user's question, producing more accurate and contextually relevant answers. [11]
Vector Database	A database designed to store, manage and index high-dimensional vector data. [12]
Embedding	Embedding is a means of representing objects like text, images and audio as points in a continuous vector space where the locations of those points in space are semantically meaningful to machine learning (ML) algorithms. [13]
Semantic Search	Semantic search is a data searching technique that focuses on understanding the contextual meaning and intent behind a user's search query, rather than only matching keywords. [14]
HTTP	The Hypertext Transfer Protocol (HTTP) is the foundation of the World Wide Web, and is used to load webpages using hypertext links. [15]
REST API (Representational State Transfer Application Programming Interface)	A REST API is an application programming interface (API) that follows the design principles of the REST architectural style. REST is short for representational state transfer, and is a

	set of rules and guidelines about how you should build a web API. [16]
Relational Database	A relational database is a type of database that organizes data into rows and columns, which collectively form a table where the data points are related to each other. [17]
MySQL	MySQL is an open source relational database management system (RDBMS) that's used to store and manage data. [18]
JSON (JavaScript Object Notation)	JSON (JavaScript Object Notation) is a lightweight data-interchange format. It is easy for humans to read and write. It is easy for machines to parse and generate. [19]
CSV (Comma-Separated Values)	CSV (Comma-Separated Values) is a simple, text-based file format used to store tabular data. [20]
Scheduling Constraints	Requirements that limit possible scheduling assignments, such as time availability, course requirements, and workload restrictions. [9]
Bias Detection	The process of identifying potentially unfair patterns or inconsistencies in AI-generated candidate evaluations. [21]
Fairness Metrics	Quantitative measures used to assess whether evaluation outcomes demonstrate potential disparities between candidate groups. [21]
Human-in-the-Loop (HITL)	Human-in-the-loop (HITL) refers to a system or process in which a human actively participates in the operation,

	supervision or decision-making of an automated system. In the context of AI, HITL means that humans are involved at some point in the AI workflow to ensure accuracy, safety, accountability or ethical decision-making. [22]
Workflow Orchestration	The coordination of tasks, information, and interactions between multiple agents or system components. [4]
Pseudonymization	De-identification technique that replaces an identifier (or identifiers) for a data principal with a pseudonym in order to hide the identity of that data principal. [23]
Natural Language Processing (NLP)	Natural language processing (NLP) is a subfield of computer science and artificial intelligence (AI) that uses machine learning to enable computers to understand and communicate with human language. [10]
Unstructured Text Data	Text-based information that has no fixed schema and can have a more complex format, such as audio files and web pages. [24]
Pandas	Pandas is a Python library used for working with data sets. It has functions for analyzing, cleaning, exploring, and manipulating data. [25]

4. Short Bios

Elias Poitras-Whitecalf	I am a fifth-year Software Engineering student at the University of Calgary. For the past four years, I have worked as an undergraduate research assistant with ADAPT Labs, focusing on engineering education, digital literacy, AI literacy, and the

	development of AI tools that promote critical thinking. I have also contributed to several research publications in these areas. My technical interests include full-stack web development, software design, and data analytics. I aspire to work as a software developer, ideally in an industry that combines technology with creativity, such as music, video games, art, or digital communication.
Sam Haque	I am a fifth-year Software Engineering student at the University of Calgary. I spent my fourth-year interning at Circle Cardiovascular Imaging as a Software Quality Assurance Intern, where I worked on cutting-edge medical software products that apply FDA-approved machine learning solutions to analyze MR and CT scans, reducing the effort required by clinicians to draw contours, among other technologies and tools. Outside of my work experience, I have built several personal software projects that have allowed me to apply my full-stack development skills, project management experience, and understanding of UI/UX design from a psychological perspective. I aspire to contribute to software solutions that directly benefit people while adequately

	considering ethical, environmental, and social concerns.
Adol Awan	I am a final-year Software Engineering student at the University of Calgary. I have spent the last two summers conducting research on adaptive learning systems and academic integrity communication patterns in the age of AI. I am also a co-author of two published research papers in these areas. My technical interests include AI safety and security, AI testing and reliability, forward deployment, and systems design.
Erioluwa Olubadejo	I am a fifth-year Software Engineering student at the University of Calgary and an undergraduate research assistant studying security risks in AI agent systems. I also have freelance full-stack development experience, having built more than ten websites for businesses and personal clients. My interests include artificial intelligence, cybersecurity, and workflow automation. My projects include an AI security scanner and an automated semester deadline tracker. I aim to develop reliable, secure software that solves practical problems.

5. Project Motivation

The current TA hiring and scheduling process relies heavily on manual data collection and evaluation. Applicant information is gathered through surveys, emails, and other resources before being organized into Excel spreadsheets. Professors and course coordinators must then manually search through and analyze these records to identify suitable candidates while also considering their qualifications, experience, availability, and course requirements.

This process is time-consuming, particularly when managing a large number of applicants across multiple courses. Additionally, manually reviewing candidate information makes it difficult to apply consistent evaluation criteria, potentially introducing human bias and inconsistencies in candidate selection. Reliance on spreadsheets also increases the risk of accidental data modifications, loss of information, and difficulties maintaining accurate and up-to-date records.

Although existing methods for tracking TA availability help simplify scheduling, professors must still dedicate significant time to determining whether candidates meet the requirements of specific courses. These limitations highlight the need for a more efficient, consistent, and centralized approach to TA evaluation and scheduling.

Project Objectives

The primary objective of this project is to develop a process for **Optimizing Placements for Teaching Assistants using Intelligent Multi-agent Analytics (OPTIMA)** that assists professors in evaluating, ranking, and assigning TA candidates to suitable courses.

The system will use a multi-agent architecture to automate several stages of the hiring and scheduling process, including extracting relevant information from applications, analyzing candidate qualifications, and generating rankings based on predefined evaluation criteria. Factors such as technical skills, previous teaching experience, academic qualifications, and candidate availability will be considered when determining suitability for specific TA positions.

The project aims to achieve the following objectives:

1. **Automate Candidate Information Processing:** Extract and organize relevant information from TA applications and survey responses to reduce manual data entry and review.
2. **Support Consistent Candidate Evaluation:** Evaluate applicants using standardized criteria to improve consistency and transparency in the ranking process.
3. **Improve Candidate-to-Course Matching:** Suggest suitable TA candidates based on their qualifications, experience, and the specific requirements of each course.
4. **Simplify TA Scheduling:** Assist in suggesting candidates to available positions while considering scheduling conflicts, availability, and course requirements.
5. **Reduce Bias and Improve Fairness:** Minimize potential bias by removing personal identifying information, such as names and gender, before candidate data is processed by the AI agents. Open-ended survey responses will also undergo keyword extraction to identify relevant skills, qualifications, and experiences while filtering out gendered language, spelling errors, and unnecessary filler words. This approach aims to ensure that candidates are evaluated primarily on their qualifications rather than personal characteristics or writing styles.
6. **Support Informed Decision-Making:** Provide professors with structured candidate rankings and relevant supporting information while maintaining human oversight of final hiring decisions.

The final product will be a centralized system that reduces the administrative workload associated with TA hiring and scheduling. Rather than replacing professors' decision-making responsibilities, the system will serve as a **decision-support tool** that improves efficiency, consistency, and accessibility of candidate information.

6. Project Scope and Deliverables

OPTIMA aims to streamline the process of evaluating, ranking, and scheduling TA. The system will use a multi-agent AI architecture to process TA application information, evaluate candidates

against course requirements, and generate ranking and scheduling recommendations for professors.

The system will function as a decision-support tool rather than an automated hiring solution. Professors will retain control over final TA selections and scheduling decisions. The project will also prioritize data privacy, compatibility with existing systems, and the reliability of AI-generated outputs.

The project scope and deliverables have been categorized using the MoSCoW prioritization method: Must-Haves, Should-Haves, Could-Haves, and Won't-Haves. These categories establish the minimum requirements for project completion while identifying additional functionality that may be implemented if time and resources permit.

6.1 Must-Have Deliverables

The user stories underlying these requirements are summarized in Fig. A4.

The following requirements are essential to the successful completion of the project.

Deliverable	Description
Data Privacy and Anonymization	Personal identifying information must not be shared with AI agents. Candidate information will be anonymized before processing to reduce privacy risks and potential bias.
Legacy System Compatibility	The application must be compatible with the existing codebase and relevant functionality of the current Schulich Studio system.
Candidate Ranking and Scheduling	The system must generate candidate rankings and scheduling recommendations based on qualifications, experience, availability, and course requirements. Professors must be presented with final recommendations.
Complete Schulich Studio System	The core Schulich Studio application must be functional and integrated by the end of the project in April 2027. A hosted website with a user-friendly interface and tools. This solution should be ready to ship and begin early practical usage.

Data Integrity	Existing candidate records must remain unchanged during AI processing and evaluation. The system must prevent unintended modifications to source data.
Multi-Agent Communication	AI agents must be able to communicate and exchange structured information to complete the ranking and scheduling workflow.
Input Validation	User input (primarily in free text user options) should be validated to ensure it doesn't include prompt/SQL injection or unsafe content.
Output Validation	AI-generated outputs must undergo validation before being presented to the end user to identify errors, inconsistencies, or invalid recommendations. Outputs should also handle irrelevant input appropriately.
Manual TA Selection	Professors must be able to manually select candidates for scheduling rather than relying exclusively on AI-generated rankings.
Explainable Candidate Rankings	The system should provide reasoning for candidate rankings, allowing professors to understand which qualifications and criteria contributed to the recommendations.
Configurable Ranking Limit	Professors must be able to specify how many candidates appear in the final ranking, such as the top 8, 10, or 12 candidates.

6.2 Should-Have Deliverables

The following features would significantly improve the system's functionality, flexibility, and usability but are not essential to its minimum operation.

Deliverable	Description
Dynamic Application Functionality	The application should support different candidate datasets, and evaluation requirements without relying on hardcoded values or being limited to a single configuration.
Adjustable Ranking Weights	Professors should be able to modify the weighting of evaluation criteria, such as experience, technical skills, and qualifications, to reflect course-specific requirements.

Configurable Agent Instructions	The system should allow authorized users to adjust instructions provided to the managing agent to accommodate different evaluation needs.
TA Preselection	Professors should be able to exclude specific TA candidates for consideration before generating final rankings or schedules if they are already scheduled or preselected to be scheduled.
Hard and Soft Constraints	The scheduling system should distinguish between hard constraints, which must be satisfied, and soft constraints, which represent preferences that should be satisfied where possible.
Keyword Extraction and Text Standardization	Open-ended TA survey responses should be processed to extract meaningful keywords and phrases, reducing the influence of irrelevant language, filler words, and spelling differences.
Ranking Re-evaluation	Professors should be able to regenerate candidate rankings and recommendations using different evaluation weightings without modifying the original candidate information.

6.3 Could-Have (Nice-to-Have) Deliverables

These features may be implemented if the core requirements are completed, and sufficient development time remains.

Deliverable	Description
Multiple Scheduling Recommendations	The system could generate up to five alternative scheduling configurations, allowing professors to compare different valid assignments.
Retrieval-Augmented Generation (RAG)	RAG could be implemented to provide AI agents with relevant external information, such as course requirements, evaluation guidelines, and historical selection criteria.

Vector Database	A vector database could be used to store and retrieve embedded text information, supporting semantic search and retrieval of relevant documents.

6.4 Out-of-Scope Features

The following features are excluded from the current project scope and are not expected to be delivered by April 2027.

Feature	Description
TA Dashboard	A dedicated dashboard for TAs to access and manage their assignments will not be developed.
TA Information Additions	Functionality allowing users to add additional TA information through the application will not be implemented.
OpenAI Agents SDK Sandbox	A dedicated sandbox environment for experimenting with AI agents will not be included.
Additional UI Enhancements	Major interface redesigns or visual enhancements beyond those required for core functionality are excluded.
Desktop and Mobile Applications	Dedicated desktop and mobile applications, as well as mobile-specific interface development, will not be included. The project will focus on the primary web application.

6.5 Final Project Deliverables

By the completion of the Winter 2027 term, the team aims to deliver the following:

1. **Functional Web Application:** An integrated Schulich Studio application that enables professors to review TA candidate rankings and scheduling recommendations.
2. **Multi-Agent AI System:** A functioning architecture consisting of specialized AI agents responsible for processing candidate information, evaluating qualifications, generating rankings, and supporting scheduling.

3. **Backend API and Database Integration:** Backend services and database functionality supporting communication between the user interface, candidate records, and AI agents.
4. **Source Code and Documentation:** The completed source code maintained in GitHub, along with technical documentation describing the system architecture, implementation, and operation.
5. **Testing and Deployment:** Test cases validating core functionality, integration, and data handling, along with CI/CD pipelines and deployment configurations.
6. **Final Demonstration:** A working demonstration of the completed system showing the candidate ranking and scheduling workflow.

6.6 Scope Management

The project's scope and deliverables were reviewed and agreed with E. Marasco and T. Al Shoura to ensure that the proposed requirements align with their expectations. The Must-Have deliverables serve as the minimum criteria for successful project completion.

Any subsequent changes to project scope, functionality, or deliverable priorities will require mutual agreement between the project team and sponsor. Proposed changes will be evaluated based on their impact on development time, available resources, technical feasibility, and existing project commitments.

The team will prioritize Must-Have requirements throughout development, followed by Should-Have and Could-Have features as time and resources permit.

7. Preliminary Design of the Proposed Solution

The proposed solution is an Agentic AI-based system designed to automate and streamline the evaluation, ranking, and scheduling of TA candidates. The system will employ a multi-agent architecture in which specialized AI agents collaborate to process applicant information, evaluate qualifications, generate candidate rankings, and recommend suitable scheduling assignments.

Although the final product is intended to function as a centralized, reusable API that could support hiring and candidate evaluation beyond the Schulich Studio TA scheduling application, the preliminary implementation will focus on establishing a functional local instance of the agent architecture. This initial implementation will demonstrate the feasibility of the proposed approach using anonymized TA application data before being integrated into the existing Schulich Studio application.

7.1 System Architecture

The preliminary design will consist of several specialized AI agents coordinated through a central workflow orchestrator. Each agent will be responsible for a specific stage of the candidate evaluation and scheduling process. Agents will communicate through structured JSON outputs to ensure that information can be consistently transferred, interpreted, and validated between components. The overall agent architecture is shown in Fig. 1.

The proposed architecture will consist of the following components:

7.1.1. Data Processing and Anonymization

TA application records supplied as CSV files will be read and standardized using Python, primarily Pandas. Before any information reaches an AI agent, it will pass through a deterministic, rule-based anonymization stage that uses no LLMs. Using an LLM to detect personal information would require sending that information to the model, which defeats the purpose of anonymization. A rule-based approach also behaves predictably and can be tested exhaustively.

Anonymization will operate in three layers:

1. **Structured fields.** Columns containing direct identifiers, such as names, UCIDs, and email addresses, will be removed and replaced with a pseudonymous identifier (e.g., CAND-0042). The mapping between identifiers and original records will be stored separately and never sent to an agent.
2. **Free-text responses.** Regular expressions will detect and replace email addresses, phone numbers, UCID-format numbers, and URLs. Each applicant's own name tokens (first, last, and preferred names, matched case-insensitively and including possessive forms) will be taken from their record and replaced wherever they appear in their responses. Gendered pronouns and honorifics will be replaced with neutral terms.
3. **Verification gate.** Immediately before any payload is sent to an LLM, it will be re-scanned using the same patterns and the full list of applicant names in the dataset. If any match is found, the request will be blocked and logged rather than sent. The system fails closed.

The original applicant records will remain unchanged, and anonymized copies will be maintained separately for AI processing. Rule-based anonymization cannot detect every indirect identifier, such as a candidate describing a unique personal circumstance. As stated in Section 9, the team therefore also recommends that the sponsor pseudonymize data before providing it to the system. A local, non-LLM named-entity recognition library may be added as a further layer if testing shows that regular expressions miss identifiers.

7.1.2. Keyword Extraction Agent

The Keyword Extraction Agent will analyze anonymized applicant responses and identify relevant qualifications, skills, experiences, and other information that may contribute to candidate suitability.

Rather than passing lengthy and potentially inconsistent survey responses directly into subsequent evaluation stages, this agent will produce a structured representation of each candidate's qualifications. Extracted information will be organized into predefined categories, allowing subsequent agents to evaluate candidates using consistent information.

For example, an applicant's response describing previous experience assisting with programming laboratories could be categorized under teaching experience and technical subject knowledge.

The agent will return its extracted information in a standardized JSON format for use by the remaining agents.

7.1.3. Candidate Evaluation Agent

The Candidate Evaluation Agent will assess each applicant against the requirements and evaluation criteria associated with a particular course or TA position.

Evaluation criteria may include:

- Previous teaching and instructional experience.
- Subject knowledge and relevant technical skills.
- Academic qualifications.
- Previous performance, where applicable.
- Other course-specific or client-defined requirements.

Each criterion will be evaluated independently, with the agent producing a criterion score and an evidence-based explanation supporting its assessment. The evaluation will use the extracted applicant information alongside the requirements provided for the position.

The evaluation process will distinguish between mandatory requirements and preferences. Mandatory requirements will determine whether a candidate is eligible for consideration, while preferences will contribute to the candidate's overall suitability score without automatically disqualifying them.

7.1.4. Candidate Ranking Agent

The Candidate Ranking Agent will use the evaluation results to calculate an overall suitability score for each eligible applicant and generate an ordered ranking.

The preliminary design will use a weighted scoring methodology in which individual criterion scores are combined according to configurable weights provided for the hiring request.

Candidates who fail mandatory requirements will be excluded before the final ranking is generated. The remaining candidates will be ordered by descending overall suitability score.

The ranking process will also support configurable output limits, allowing professors to request a specific number of candidates, such as the top 8, 10, or 12 applicants.

To maintain consistency and numerical accuracy, the weighted score calculation will be performed using deterministic application logic rather than relying exclusively on an LLM to perform mathematical calculations. The AI agents will provide the qualitative evaluations and supporting evidence, while the ranking component will apply the predefined scoring methodology.

7.1.5. Scheduling Agent

The Scheduling Agent will generate recommended TA-to-section assignments from the shortlists produced by the Candidate Ranking Agent. Its inputs will include the eligible, ranked candidates for each course, the sections or time slots that need TA coverage, each candidate's availability, workload limits (such as maximum weekly hours or number of assignments per term), any candidates the professor has preselected or excluded, and the constraints configured for the request.

Hard constraints are conditions that every assignment must satisfy. They include no overlapping time commitments, assignment only within a candidate's stated availability, adherence to workload limits, and exclusion of candidates who failed a course's mandatory requirements. Soft constraints are preferences that should be satisfied where possible, such as favoring higher-ranked candidates, honoring candidates' course preferences, and distributing assignments evenly.

The agent will return a proposed schedule in a predefined JSON schema. For each assignment, the schema will contain the candidate's pseudonymous identifier, the course and section, and a brief rationale referencing the rankings and constraints that informed the choice. Sections that cannot be filled without violating a hard constraint will be listed explicitly as unfilled rather than assigned to an unsuitable candidate.

Because LLMs cannot guarantee strict constraint satisfaction, the agent's output will not be trusted directly. A deterministic schedule validator, implemented in application code, will check every proposed assignment against the hard constraints. If it finds violations, they will be returned to the Scheduling Agent for revision, up to a fixed number of attempts (e.g., three). If a valid schedule still cannot be produced, the system will report the unresolved conflicts to the professor instead of returning an invalid schedule, consistent with the failure-handling approach in Section 7.4. The validator will also score soft-constraint satisfaction, so professors can see how well each schedule meets the stated preferences.

During the preliminary implementation, the Scheduling Agent and validator will be developed locally using anonymized or synthetic availability and section data, formatted to match the existing Schulich Studio records. Integration with "live" Studio scheduling data will take place in Sprint 2, once the team incorporates the existing architecture. The same design supports the Could-Have feature of multiple alternative schedules: the agent can be asked for several configurations, each validated independently.

7.1.6. Manager Agent and Workflow Orchestration

A central Manager Agent will coordinate the overall evaluation workflow and oversee communication between specialized agents.

The Manager Agent will be responsible for receiving the initial evaluation request, distributing relevant information to the appropriate agents, collecting their outputs, and ensuring that the workflow progresses in the intended sequence.

It will also consolidate the results into a final response containing candidate rankings, scheduling recommendations, and supporting explanations.

Where necessary, the Manager Agent will identify incomplete or inconsistent agent outputs and initiate additional evaluation or validation steps. It will not independently override mandatory requirements or replace the deterministic scoring and validation logic.

7.2 Proposed System Workflow

The system will follow a sequential workflow in which applicant information is progressively processed and evaluated before final recommendations are generated.

The proposed workflow is as follows (Fig. 2 illustrates this workflow as an activity diagram):

1. **Input:** The system receives anonymized TA application data, course requirements, evaluation criteria, criterion weights, scheduling information, and the requested ranking limit.
2. **Data Processing:** Applicant records are read from CSV files, standardized, and checked for completeness. Personal identifying information is removed before the data is provided to AI agents.
3. **Information Extraction:** The Keyword Extraction Agent identifies relevant skills, qualifications, teaching experience, and other information from applicant responses.
4. **Candidate Evaluation:** The Candidate Evaluation Agent evaluates applicants against the specified criteria and produces individual criterion scores with supporting explanations.
5. **Candidate Ranking:** The Candidate Ranking Agent applies the configured weighting methodology, excludes candidates who fail mandatory requirements, and generates an ordered shortlist.
6. **Scheduling:** The Scheduling Agent proposes assignments from the ranked shortlists, which the deterministic validator checks against hard constraints.
7. **Output Validation:** The resulting rankings and scheduling recommendations undergo validation to identify missing information, invalid assignments, inconsistent scores, or improperly structured outputs.
8. **Final Output:** The Manager Agent consolidates the validated results into a structured JSON response containing candidate rankings, suitability scores, supporting explanations, and scheduling recommendations.
9. **Human Review:** Professors review the recommendations and retain full authority over final candidate selections and scheduling decisions.

7.3 Communication and Data Exchange

The multi-agent architecture will primarily be developed using Python and the OpenAI Agents SDK. The SDK will provide the underlying infrastructure for defining specialized agents, configuring their instructions, and coordinating interactions between them.

JSON will serve as the primary data exchange format between agents. Each agent will be designed to accept structured input and return outputs following predefined schemas. This approach will reduce ambiguity during agent communication and simplify integration with the backend API in subsequent development stages.

Pandas will be used to process and manipulate CSV-based applicant records before they enter the agent workflow. The preliminary implementation will operate locally, allowing the team to develop and test the architecture independently of the final web application.

As development progresses, the agent workflow will be exposed through a REST API. This will allow the existing Schulich Studio frontend and backend to submit evaluation requests and receive structured ranking and scheduling results without needing to interact directly with individual AI agents.

7.4 Data Privacy, Validation, and Human Oversight

Data privacy and output reliability will be fundamental considerations throughout the proposed design.

Applicant information will be anonymized before being transmitted to AI models. The system will use pseudonymous candidate identifiers to maintain consistency between evaluation results and the original applicant's records without exposing personal identifying information to the agents.

The existing sponsor database will not be modified during AI processing. Applicant information will be treated as read-only input, and generated evaluations and recommendations will be maintained separately from the original records.

Additionally, AI-generated outputs will undergo validation before being returned to the application. Validation will include checking JSON structure, required fields, criterion score ranges, ranking order, requested candidate limits, and scheduling constraints. If a mandatory criterion cannot be evaluated because of missing information or an agent failure, the system will explicitly report the failure rather than produce an unsupported successful ranking.

Finally, the system will operate as a human-in-the-loop decision-support tool. AI-generated rankings and schedules will serve as recommendations rather than automatic hiring decisions. This will be emphasized in the displayed messaging to the user. Professors will be able to review candidate evaluations, consider supporting explanations, manually select candidates, and make final scheduling decisions.

7.4.1: Prompt and SQL Injection Prevention

Prompt injection is ranked as the most critical risk in the OWASP Top 10 for LLM Applications [26] - SQL Injection having a similar attack surface. OPTIMA has two injection surfaces. The first is direct: professors' free-text inputs, such as custom criteria and agent instructions. The second, and more significant, is indirect: applicant survey responses, which applicants write themselves and which the agents then process. Indirect prompt injection, where instructions are planted in data an LLM application later reads, has been demonstrated against real LLM-integrated systems [27]. Benchmarking studies have found that no single existing defense is fully effective [28]. OPTIMA therefore will use layered defenses aligned with the OWASP mitigation guidance [26]:

- **Constrained agent behavior.** Each agent's system instructions will define its role and limits and state that applicant content is data to be evaluated, never instructions to follow.
- **Input screening.** Before evaluation, free text will be checked with pattern-based filters for instruction-like content (e.g., "ignore previous instructions") and for excessive length. Flagged responses will still be evaluated, but they will be marked for professor review rather than silently dropped.
- **Structured, validated outputs.** Every agent returns JSON that is validated against a schema by deterministic code. Scores outside permitted ranges or malformed outputs are rejected.

- **Validation at every step.** In the OpenAI Agents SDK, input guardrails run only for the first agent and output guardrails only for the agent producing the final output [29]. Validation will therefore be applied at each agent handoff, not only at the workflow boundaries.
- **Least privilege.** Agents will have no database write access and no tools beyond what their task requires.
- **Human oversight.** Professors make all final decisions, so a successful injection cannot directly cause a hiring outcome.

7.4.2 Bias Mitigation

LLM-based screening tools have been shown to reproduce demographic bias. In an audit of language-model resume screening, Wilson and Caliskan found significant preferences based on the race and gender signals carried by candidates' names [30]. Bias can also enter through writing style. In academic hiring, Madera et al. found that women were described in more communal terms than men, and that communal descriptions were associated with lower hiring evaluations [31]. NIST's guidance on AI bias states that zero bias risk is not achievable, so bias must be managed and measured throughout the system lifecycle [32]. OPTIMA therefore combines several mitigations with explicit testing:

- **Removal of identity signals.** The anonymization stage (Section 7.1.1) removes names, pronouns, and honorifics before any AI processing, which addresses the name-based bias identified in [30].
- **Reduced influence of writing style.** The Keyword Extraction Agent converts free-text responses into standardized qualification categories, so candidates are evaluated on qualifications rather than tone or phrasing [31].
- **Criterion-based evaluation.** Candidates are scored on each criterion independently with supporting evidence, rather than receiving a single holistic judgment, which makes inconsistent or unjustified scores easier to detect.
- **Mitigation of position bias.** LLM evaluators have been shown to favor candidates based on their order of presentation [33]. Each candidate will therefore be evaluated in a

separate request rather than in a single list, candidate order will be randomized, and the final ranking will be computed by deterministic code.

- **Human oversight.** Rankings are presented as recommendations with explanations, and professors retain all final decisions.

7.5 Preliminary Implementation and Future Expansion

The preliminary implementation will prioritize demonstrating the feasibility of the multi-agent architecture rather than delivering every feature of the final application.

The initial prototype will operate locally using anonymized TA application records and predefined evaluation criteria. Its primary objective will be to demonstrate that the agents can successfully communicate, extract relevant candidate information, evaluate qualifications, generate consistent rankings, and produce preliminary scheduling recommendations.

Following successful validation of the MVP, the architecture will be expanded into the complete Schulich Studio application. This will involve integrating the agent workflow with a Python backend, REST API endpoints, database functionality, and the existing web interface.

The system will also be designed to support configurable evaluation criteria and weights, allowing it to accommodate different courses and potentially other hiring scenarios without requiring modifications to the underlying ranking logic.

Additional functionality, including explainable rankings, configurable agent instructions, alternative scheduling configurations, and Retrieval-Augmented Generation (RAG), may be introduced as development progresses and available resources permit.

Overall, the proposed design establishes a modular and extensible multi-agent architecture that separates applicant processing, candidate evaluation, ranking, and scheduling into independently manageable components. This approach is intended to improve the consistency, transparency, and efficiency of TA hiring while maintaining data privacy and ensuring that final decisions remain under human control.

FIGURE 1. AGENTIC AI ARCHITECTURAL DIAGRAM

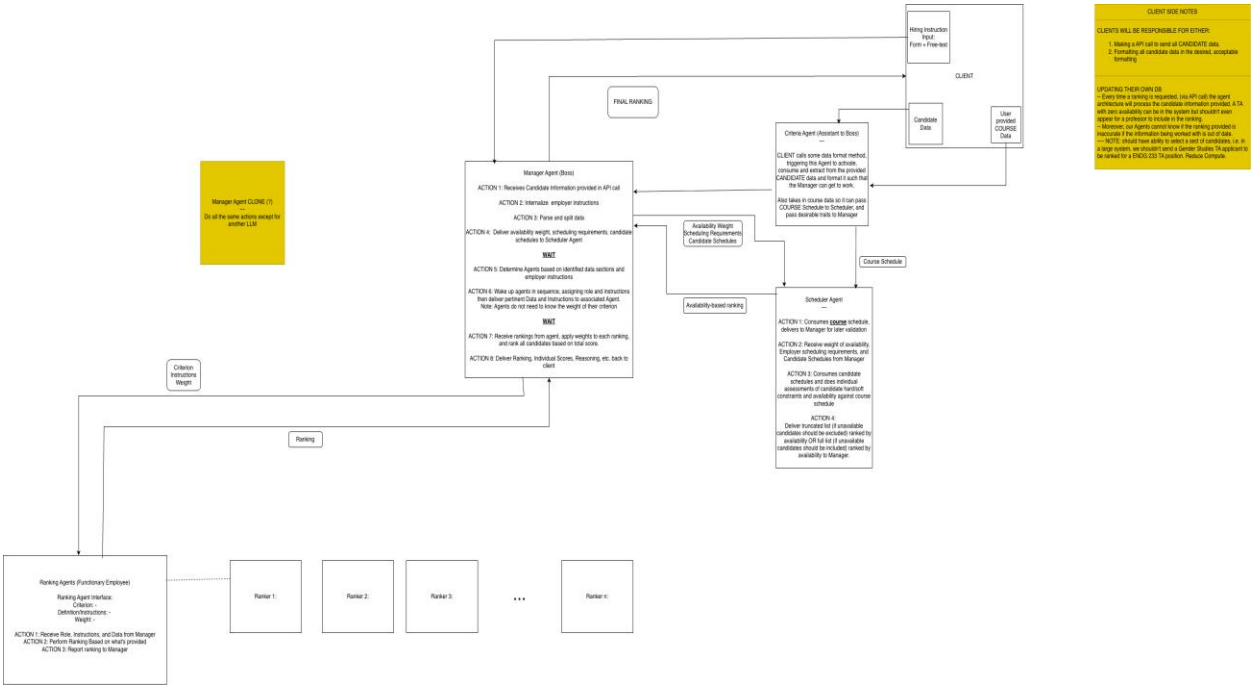


FIGURE 2. ACTIVITY DIAGRAM

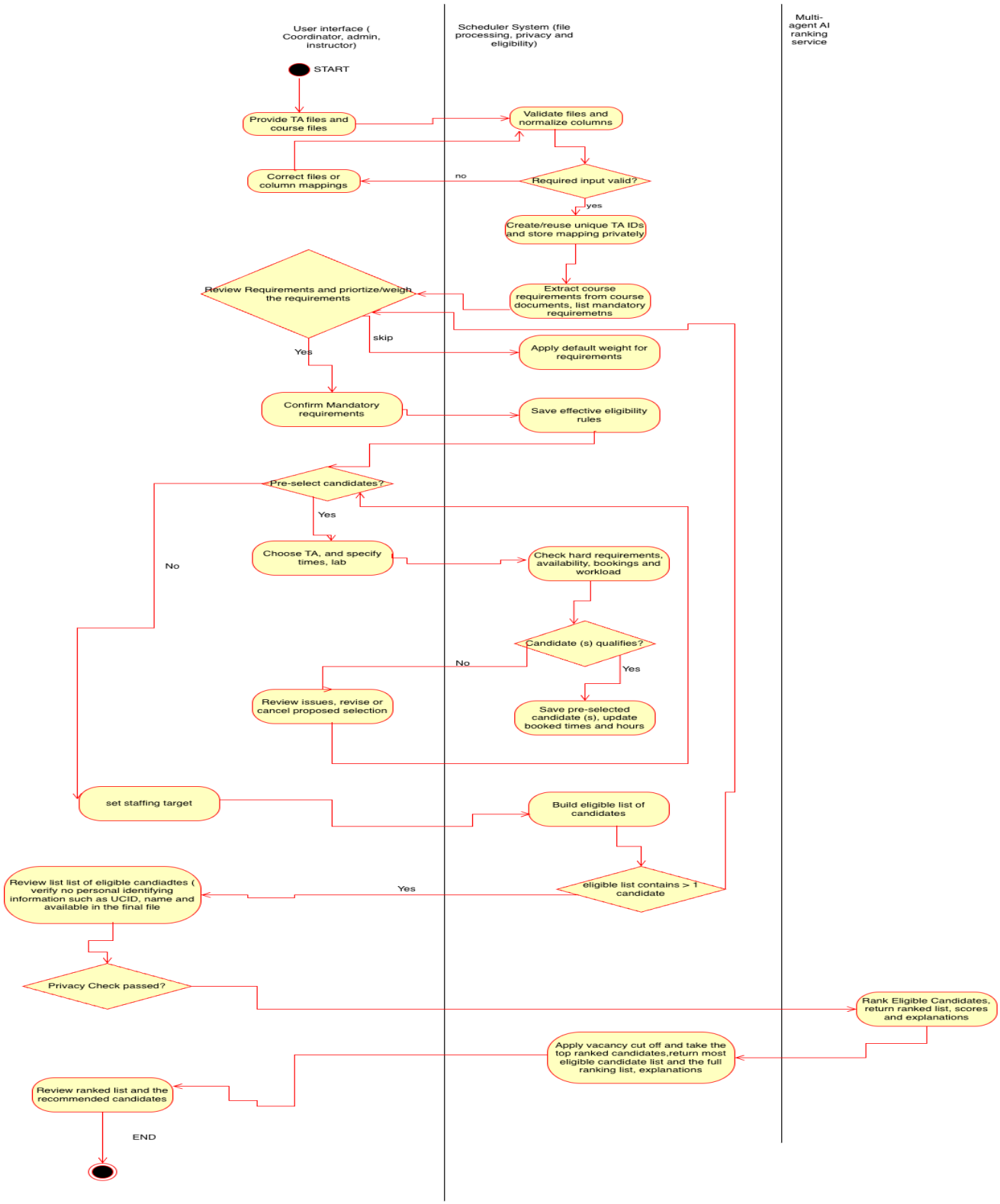
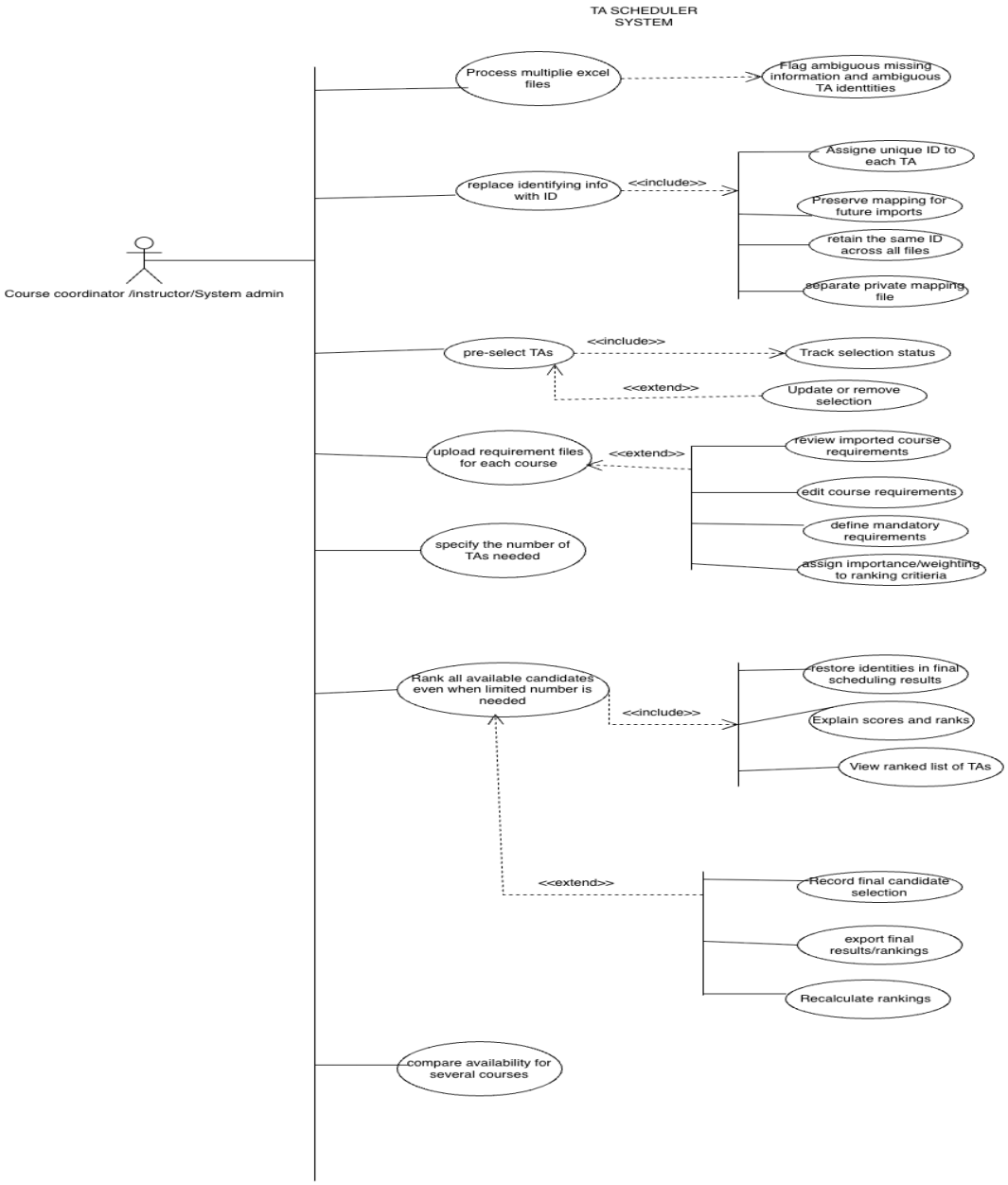


FIGURE 3. USE-CASE DIAGRAM


TA SCHEDULER SYSTEM
Process multiplie excel files
Flag ambiguous missing information and ambiguous TA identtities
replace identifying info with ID
<<include>>
Assigne unique ID to each TA
Preserve mapping for future imports
retain the same ID across all files
separate private mapping file
Course coordinator /instructor/System admin
pre-select TAs
<<include>>
Track selection status
<<extend>>
Update or remove selection
upload requirement files for each course
<<extend>>
review imported course requirements
edit course requirements
define mandatory requirements
assign importance/weighting to ranking critieria
specify the number of TAs needed
Rank all available candidates even when limited number is needed
<<include>>
restore identities in final scheduling results
Explain scores and ranks
View ranked list of TAs
<<extend>>
Record final candidate selection
export final results/rankings
Recalculate rankings
compare availability for several courses

8. Alternative Solutions/Methodologies

8.1 Alternative Solution: Traditional Machine Learning Models

An alternative solution considered was the use of multiple machine learning models, such as logistic regression or gradient-boosted trees, combined with a Large Language Model (LLM) or Natural Language Processing (NLP) techniques to process unstructured text data. These models could evaluate candidate qualifications, experience, availability, and course requirements to predict candidate suitability and generate rankings.

Although this approach is technically feasible, it presents several challenges specific to our project, particularly regarding the availability and quality of training data. The dataset provided by our sponsor contains anonymized applicant information, survey responses, final TA selection records, and course information. While these records could be used to train supervised machine learning models, they may not contain sufficient course-specific labels or consistently documented evaluation criteria to reliably predict candidate suitability. Mohammed et al. [34] examined the relationship between data quality and machine learning performance, highlighting how incomplete, inaccurate, or inconsistent data can negatively affect model reliability. This study reinforced the importance of ensuring that historical TA application records contain sufficient and reliable information before using them to train predictive models.

Additionally, historical hiring decisions may have been influenced by factors beyond the qualifications recorded in the dataset, such as scheduling constraints, course-specific needs, or individual preferences. Consequently, using previous selection outcomes as training labels could introduce inconsistencies or reinforce existing biases.

Traditional machine learning models would also require additional preprocessing, feature engineering, training, and validation [35], [36]. Changes to the application format, survey questions, or evaluation criteria may require adjustments to the preprocessing pipeline, input features, or model retraining to incorporate new information.

For these reasons, the team selected an Agentic AI approach as the preferred solution. By using LLM-based agents, the system can process unstructured candidate information and apply

configurable evaluation criteria without requiring a separately trained supervised prediction model. Unlike a traditional predictive model, an agentic system can coordinate multiple components and adapt its workflow to changing requirements [37]. This flexibility is particularly beneficial for our project, where course requirements and evaluation criteria may vary. However, AI-generated recommendations will still require validation to ensure consistency, accuracy, and fairness.

Another consideration was the project's development timeline and available resources. Although logistic regression and gradient-boosted trees are computationally efficient, implementing and validating these models would require additional time for data preparation, feature engineering, model training, and hyperparameter tuning. The team would also need to establish how historical selection records could accurately represent candidate suitability across different courses.

Using existing LLMs allows the team to focus more development time on the multi-agent architecture, system integration, and scheduling functionality. While traditional machine learning remains a viable alternative if sufficient reliable training data becomes available, the team considers Agentic AI to be the more practical approach given the project's current dataset, requirements, and development timeline.

8.2 Alternative Solution: AI Model Selection

At the time of writing this Design Roadmap, we are still evaluating which AI models will be most suitable for the system. Currently, the OpenAI Agents SDK [6] is being used to develop and coordinate the agents, with OpenAI models serving as the initial models.

The team plans to evaluate models from three major providers: OpenAI's GPT-6 series [38], Anthropic's Claude 5 series [39], and Google's Gemini 4 series [40]. Both the OpenAI Agents SDK's built-in model configuration capabilities and LangChain [41] will be considered for testing models across different providers.

Each model will be tested using identical instructions, anonymized TA application records, and expected output formats. Performance will be evaluated based on output accuracy, consistency,

token usage, API cost, and response time. Vendor-reported results, such as those in Fig. A1, suggest models differ substantially in hallucination rate and token use, which the team's own testing will verify.

Models with different capabilities and pricing structures will be considered for individual agent responsibilities. For example, smaller and less expensive models may be sufficient for keyword extraction and structured data processing, while more capable models may be required for complex candidate evaluation and reasoning tasks.

The final model selection will be based on experimental results to identify an appropriate balance between performance, reliability, feedback from industry sponsors, and operating costs. Depending on the results, different models may be assigned to individual agents. This approach will allow the team to make informed, data-driven decisions while remaining within the project's estimated budget.

8.3 Alternative Solution: Single-Agent vs. Multi-Agent Architecture

Another alternative considered was using a single AI agent to handle the entire TA evaluation and scheduling process rather than dividing responsibilities among multiple specialized agents.

A single-agent approach would simplify development and reduce the complexity of communication between components. However, combining keyword extraction, candidate evaluation, ranking, and scheduling into one agent could make it more difficult to independently test, validate, and modify individual stages of the workflow.

The team selected a multi-agent architecture to separate these responsibilities into specialized components. This approach allows individual agents to be developed and tested independently, supports modular development, and provides greater flexibility when modifying specific parts of the evaluation process. Although this introduces additional architectural complexity and

potentially higher API costs, the team believes the benefits of modularity and maintainability justify the approach.

The multi-agent approach also aligns with the team's development structure. By separating the system into specialized components, team members can work on different agents and their supporting functionality simultaneously [42]. The decision criteria applied by the team follow the framework in Fig. A2. This allows development and testing activities to proceed in parallel, potentially reducing the time required to implement the system.

However, the team recognizes that a multi-agent system introduces additional complexity, including communication between agents, increased API usage (and thus increased time per user request, cost per user request, etc.), and the possibility of errors being passed between components [42]. To address these challenges, agents will exchange structured data, and outputs will be validated before being used by subsequent components.

The team determined that the benefits of modularity, independent testing, and flexibility outweigh the additional implementation complexity for the proposed system. Moreover, to ensure reasonable computational time, we will experiment with configuring agents with more complex tasks to utilize more advanced models and vice versa.

8.4 Alternative Solution: Constraint Solver vs. LLM Scheduling Agent

A constraint-programming solver, such as Google OR-Tools [9], was considered for scheduling. A solver guarantees that hard constraints are satisfied, can find optimal assignments, and is fully deterministic. However, every constraint and preference must be formally modeled in advance. This makes it harder to accommodate the varied, often free-text preferences professors express, and a solver does not produce natural-language rationales for its assignments. The team selected a hybrid approach: an LLM agent proposes schedules and explains them, and a deterministic validator enforces the hard constraints. This keeps the flexibility of the agentic design without trusting the LLM with constraint satisfaction. If Sprint 1 testing shows a high validation failure rate, the team will adopt a constraint solver for assignment generation and keep the LLM only for explanation.

8.5 Existing Technology in This Field: Ashby

An existing solution in the field of AI-assisted recruitment is Ashby [43], a commercial applicant tracking and recruitment management platform. Ashby provides tools for managing applicants, evaluating candidate qualifications, scheduling interviews, and analyzing recruitment data. Its AI-assisted application review functionality allows organizations to define specific hiring criteria and automatically assess candidates against those requirements, providing supporting explanations for its evaluations.

Ashby demonstrates how AI can be used to streamline recruitment processes and reduce the manual effort required to evaluate large numbers of applicants. However, Ashby is primarily designed for general recruitment rather than academic TA selection and scheduling. While it supports candidate evaluation and interview scheduling, it does not directly address the specific requirements of assigning TAs to university courses based on qualifications, availability, and course-specific constraints.

Another consideration is cost. Ashby's Foundations plan starts at $400 USD per month, introducing a recurring expense that exceeds the proposed development budget for our project.

Although Ashby offers functionality similar to our proposed system, developing OPTIMA allows the team to focus specifically on TA ranking and scheduling, integrate with the existing Schulich Studio application, and implement our proposed anonymized candidate evaluation process. For these reasons, the team considers a custom solution more suitable for the project's requirements, available resources, and budget.

9. IP and Other Legal Agreements

Data Privacy and Protection

The project will not submit personally identifiable TA information to the AI agent architecture. This includes, but is not limited to, TA names, identification numbers, and email addresses. Before information is processed by the AI agents, non-AI mechanisms will be used to

pseudonymize relevant candidate data by replacing identifying information with unique identifiers.

Although the system will implement these mechanisms, we recommend that data be pseudonymized before being provided to the application as an additional privacy safeguard. Any information submitted for AI processing should be reviewed to ensure that direct personal identifiers have been removed, including identifying details that may appear in free-text responses. The original TA records will remain separate from the pseudonymized data used by the AI agents.

Intellectual Property Ownership

The proposed ownership arrangement distinguishes between the existing Schulich Studio TA scheduling website and the API developed as part of this capstone project.

Schulich Studio should retain ownership of its existing TA scheduling website, including its existing codebase, functionality, and related materials. The API developed by the capstone group for this project should ultimately belong to the capstone group, including the source code and implementation developed specifically for the API.

The ownership arrangement, including any rights required for Schulich Studio to integrate and use the API with its existing website, should be formally discussed and agreed upon by the capstone group, Schulich Studio, and the university before project completion. Any applicable institutional intellectual property policies or agreements will also need to be considered.

10. Technical Specifications

As a software-only project, OPTIMA does not require dedicated hardware procurement. These specifications define observable software behaviour, interfaces, data protection, and resource limits. Values labelled Proposed are engineering targets to validate with the sponsor after initial benchmarking, rather than confirmed performance results.

Specification	Measurable requirement for the final product	Verification method
API interface	Accept hiring criteria, criterion weights, and the requested number of candidates through a REST API; return results in JSON.	Submit valid requests and validate responses against the documented schema.
Candidate ranking	Return the requested number of unique candidates, or all eligible candidates if fewer are available, ordered by descending overall score.	Compare returned rankings with expected results for controlled datasets.
Configurable criteria	Support teaching experience, subject knowledge, past performance, and up to 10 client-defined criteria per request.	Submit requests containing each supported criterion and 10 custom criteria.
Weighted scoring	Calculate overall scores using the normalized weighted average of criterion scores to generate a final output.	Compare results against independently calculated reference scores.
Hard constraints	Include zero candidates who violate a mandatory requirement, including required schedule availability.	Run test cases containing eligible, ineligible, and missing data candidates.
Soft constraints	Allow preferences to affect candidate scores without automatically excluding candidates.	Test candidates with full, partial, and no satisfaction of each preference.
Explanations	Provide a criterion score and evidence-based rationale for every scored criterion for every returned candidate.	Check response completeness and review rationales against applicant records.
Ranking quality	**Proposed:** achieve at least 80% overlap with an expert-approved top-10 shortlist, averaged across at least 10 held-out hiring cases.	Compare system and expert shortlists using the same applicant data and criteria.
Ranking consistency	**Proposed:** retain at least 90% of the top-10 candidates across five identical runs of each evaluation case.	Repeat requests with unchanged data, criteria, and model configuration.

Capacity and response time	**Proposed:** process 100 applicants against five active criteria within 60 seconds for at least 95% of 20 benchmark requests.	Measure end-to-end latency under a documented hosting and model configuration.
Operating-system compatibility	Support API access from both Windows and macOS clients, with all core API acceptance tests passing on both.	Run the same client test suite on each operating system.
Applicant privacy	Expose zero direct applicant identifiers to LLMs, including identifiers embedded in free-text fields.	Inspect outbound payloads using test records containing known identifiers.
Database protection	Perform zero insert, update, or delete operations on the sponsor's MySQL database.	Verify read-only database permissions and inspect query activity.
Input Validation	Validate request schemas and reject malformed input. Use parameterized SQL and keep applicant text separate from trusted agent instructions. Proposed: pass every case in an agreed prompt-injection regression suite without unauthorized tool use or disclosure.	Run schema, SQL-injection, and prompt-injection tests, including malicious instructions embedded in CSV free-text fields. Record the test set and observed failures.
Access control	Reject 100% of requests to protected endpoints that contain missing or invalid credentials.	Execute authentication tests.
Failure handling	Explicitly report every failed criterion evaluation; return no successful ranking if a mandatory constraint cannot be evaluated.	Simulate model failures, timeouts, and invalid outputs.
Reusability	Complete at least one non-TA hiring scenario by changing configuration and criterion definitions without modifying core ranking code.	Demonstrate a second hiring scenario.

Total budgetary limit	Total project expenditure, including LLM usage, hosting, databases, and paid software, shall not exceed **CAD $1600**	Track cumulative expenditure against invoices and usage records.
Operating cost	Proposed: target no more than CAD $1.00 in metered LLM charges per complete 100-applicant ranking workflow, including all agent calls and retries. Verification of the target cost after Sprint 1 benchmarking.	Record input, output, and billable reasoning tokens where applicable, retries, model version, and provider rates for each workflow. Report mean and 95th-percentile cost in CAD.

11. Materials, supplies, tools and cost estimates

Tool / Resource	Purpose	Estimated Cost (CAD)
OpenAI API / Agents SDK	CAD $84 usage allowance for development and validation; SDK has no separate allocation. Meter all agent calls and retries.	$84.00
Claude Team subscription	Budget allowance: four standard seats × USD $25/seat/month × 6 months × 1.40 CAD/USD.	$840.00
Supabase and sponsor MySQL	Supabase Free development store Retain read-only access to sponsor MySQL. Confirm access, compatibility, and free-tier limit.	$0.00
Vercel Pro	One paid deploying seat × USD $20/month × four months × 1.40 CAD/USD. Extra paid seats and overages excluded.	$112.00
Render backend	One USD $7/month service × four months × 1.40 CAD/USD; confirm instance capacity and workspace charges before deployment.	$39.20

Domain name	Optional annual registration allowance retained from the original estimate. Use an existing sponsor domain or provider URL if available.	$21.00
Python / React / VS Code	Existing development stack and editor; no new paid licences assumed.	$0.00
GitHub and CI/CD	Repository, pull requests and automated tests within included quotas. Use mocked LLM responses for routine CI; reserve live calls for integration checks.	$0.00
Jira and team coordination	Existing team board or eligible free plan; confirm account access and limits.	$0.00
Testing data and hardware	Sponsor-approved pseudonymized or synthetic records and existing computers. Student labour is an in-kind contribution, excluded from cash expenditure.	$0.00

Cross-provider model evaluation	Planning allowance for comparative OpenAI, Anthropic and Google API trials under Section 8.2; separate from the coding-assistant subscription.	$50.00
Optional RAG and embeddings	Capped allowance only if Could-Have features proceed; use existing storage where feasible. This is not a commitment to a paid vector database.	$30.00
Deployment and reliability reserve	Planning allowance for backup exports, restricted logs, storage/egress, CI artifacts, or temporary staging compute; allocate after measuring use.	$50.00
Subtotal before contingency	All values CAD; vendor charges converted using the planning exchange rate.	$1,326.20
Contingency	15% of CAD $1,326.20; reserve for tax, exchange-rate movement, and usage variance.	$198.93
Total proposed cash budget	Includes six months of Claude Team and four months of paid hosting.	$1,525.13

Cost Estimation and Tolerance

The revised planning estimate is CAD $1,326.20 before contingency. Applying ±15% (± CAD $198.93) gives a range of CAD $1,127.27–$1,525.13. The budget including the 15% contingency is CAD $1,525.13.

Claude Team is budgeted for six months for four team members to cover the full capstone subscription period. Paid web hosting is budgeted for four months, provisionally January–April 2027. USD services use an assumed planning rate of CAD $1.40 per USD, not a quoted spot exchange rate. Confirm the deployment start date, seat requirements, regional prices, taxes, and sponsor-provided resources before purchase.

The API amounts are spending allowances rather than measured forecasts. Estimate each workflow as the sum, across all agents and retries, of billable tokens multiplied by the selected model's rates. Log actual costs during Sprint 1, cap retries and agent turns, and revise the allowance using the expected number of development, regression, and professor-validation runs. Existing personal computers will be used; no dedicated GPU or other hardware purchase is planned.

Deployment checks should cover authenticated access, secrets, TLS, privacy-safe logs, restore testing, bandwidth, CI usage, and any extra worker or staging service. Use included capabilities first and charge any paid additions against the deployment reserve. Vercel Hobby is restricted to non-commercial personal use, so do not assume it is suitable for the sponsor deployment. Recurring costs after April 2027 and responsibility for service ownership remain to be agreed upon with the sponsor.

12. Project Delivery Risks and Mitigation Strategies

The development of the Agentic AI Teaching Assistant Ranking and Scheduling System involves several potential risks that could impact the project's delivery schedule. As the project is primarily software-based and relies on online development tools and services, physical material procurement and shipping delays are not expected to pose significant risks. However, delays in obtaining sponsor resources, technical integration difficulties, and team member availability could affect project progress.

The following table identifies the primary project delivery risks and the strategies that will be used to mitigate their impact.

Risk	Potential Impact	Mitigation Strategy	Ownership
Delayed Sponsor Resources	Delays in receiving TA application data, existing source code, or additional requirements could prevent development and testing from progressing as scheduled.	Request resources early and maintain regular communication with the sponsor. Use anonymized historical or synthetic datasets for development when necessary.	Current Team Lead
Team Member Unavailability	Illness, personal circumstances, or academic commitments could delay individual responsibilities and affect sprint deadlines.	Maintain documentation and shared code repositories to allow responsibilities to be redistributed. Team members will communicate availability issues as early as possible.	Current Team Lead

System Integration Difficulties	Integrating the multi-agent architecture with the backend, database, and frontend may introduce unexpected technical challenges and require additional development time.	Establish API specifications and standardized data formats early. Perform incremental integration testing throughout development to identify issues before the final sprint.	Adol and Elias
Technical Limitations	Limitations or unexpected behavior in the OpenAI Agents SDK, selected AI models, or hosting services could require changes to the planned implementation.	Conduct preliminary testing and research during early development. Maintain alternative implementation approaches to reduce dependence on any single technology.	Sam and Erioluwa
Scope Changes	Additional requirements or changes to existing deliverables could increase the workload and delay completion of essential functionality.	Follow the agreed-upon project scope and prioritize Must-Have deliverables. Any changes to requirements will require mutual agreement between the sponsor and project team.	Current Team Lead
Development Delays	Underestimating the complexity of implementation, debugging, or testing	Track development progress through Jira, review sprint objectives regularly, and allocate sufficient time for testing	Current Team Lead

	could result in missed milestones.	and resolving technical issues.	

Risk Monitoring and Management

The team will actively monitor project delivery risks throughout the three development sprints. Risks and development blockers will be discussed during team meetings and tracked through Jira when necessary. The rotating Team Lead will be responsible for monitoring potential delays and coordinating mitigation efforts.

If a risk threatens the completion of a sprint milestone, the team will reassess task priorities, redistribute responsibilities, or adjust the implementation approach while preserving the agreed-upon Must-Have deliverables.

By identifying potential issues early and maintaining flexibility in task allocation and implementation methods, the team aims to minimize delays and ensure the project is completed by April 5, 2027.

13. Major Technical Tasks and Milestones

The development of the Agentic AI Teaching Assistant Ranking and Scheduling System will be divided into three major sprints, following the timeline established by the capstone course. Each sprint will focus on a different stage of development, progressing from the initial multi-agent architecture to a fully functional and tested application.

13.1 Project Milestones

Sprint	Target Date	Major Technical Tasks	Expected Deliverables
Sprint 1: Agent Architecture and MVP	December 4, 2026	Design and implement the multi-agent architecture, develop agents for keyword extraction, candidate evaluation, ranking, and scheduling, and establish communication between agents. Conduct initial testing using anonymized TA application data.	Functional Minimum Viable Product (MVP) demonstrating the core agent architecture and its ability to process TA information and generate preliminary rankings and scheduling results.
Sprint 2: Website Development and API Integration	February 5, 2027	Develop the frontend and backend of the web application, implement database functionality, create API endpoints, and integrate the multi-agent system with the website. Test communication between the application and AI agents.	Functional web application is connected to the multi-agent system through an API, allowing users to interact with the system and view candidate information, rankings, and scheduling results.

Sprint 3: System Completion, Testing, and Deployment	April 5, 2027	Complete remaining system functionality, implement automated test cases, develop CI/CD pipelines, apply DevOps practices, perform integration testing, and resolve outstanding issues. Finalize documentation and prepare the system for demonstration.	Fully integrated and tested TA ranking and scheduling system with automated testing, CI/CD pipelines, deployment configurations, and completed project documentation.

13.2 Sprint 1: Agent Architecture and MVP

Timeline: Design Roadmap submission – December 4, 2026

Goals and Expected Outcomes

The primary goal of Sprint 1 is to establish the core multi-agent architecture and develop a functional MVP. This includes designing the responsibilities of each agent, implementing communication between agents, and developing the initial candidate processing, ranking, and scheduling functionality.

By the end of the sprint, the system should be capable of accepting anonymized TA application data, extracting relevant qualifications and experiences, and producing preliminary candidate rankings and scheduling recommendations.

Evaluation Criteria

The MVP will be evaluated based on the following criteria:

- The system can successfully read and process TA application data from CSV files.
- The keyword extraction agent can identify relevant skills, qualifications, and experiences.
- Candidate ranking and scheduling agents can generate preliminary outputs based on predefined criteria.

- Agents can successfully exchange information and complete the intended workflow.
- Personal identifying information is excluded from the data provided to the AI agents.

Technical Specifications, Tools, and Resources

Development will primarily use Python and the OpenAI Agents SDK to implement and coordinate the AI agents. Candidate information will be processed using libraries such as Pandas, with JSON used to exchange structured information between agents. Existing anonymized TA application records will be used for development and testing.

The estimated cost of Sprint 1 is approximately **$137 CAD**, primarily allocated to OpenAI API usage during agent development and testing as well as a Team Claude Plan for general development assistance.

Alternative Solutions

Alternative approaches include using a single AI agent rather than a multi-agent architecture or implementing a traditional rule-based evaluation system. The multi-agent approach was selected to separate responsibilities into specialized components, allowing individual agents to be developed, tested, and improved independently.

13.3 Sprint 2: Website Development and API Integration

Timeline: December 2026 – February 5, 2027

The second sprint will focus on developing the web application and integrating it with the multi-agent system developed during Sprint 1.

Major tasks include implementing the frontend interface, developing the Python backend, configuring the database, creating API endpoints, and integrating the agent architecture with the website.

The expected outcome is a functional application that allows professors to interact with the TA ranking and scheduling system through a centralized interface.

Prerequisite: Completion of the core multi-agent architecture and successful preliminary testing during Sprint 1.

13.4 Sprint 3: Final System, Testing, and Deployment

Timeline: February 2027 – April 5, 2027

The final sprint will focus on completing the remaining system functionality, improving reliability, and preparing the project for final demonstration.

Major tasks include implementing comprehensive test cases, performing integration and end-to-end testing, establishing CI/CD pipelines, applying DevOps practices, validating candidate data processing and bias mitigation measures, and resolving outstanding issues.

The expected outcome is a fully functional, integrated, and tested TA ranking and scheduling system ready for deployment and final presentation.

Prerequisite: A functional website with established API communication and integration with the multi-agent system from Sprint 2.

14. Roles and Responsibilities

The development of the Agentic AI Teaching Assistant Ranking and Scheduling System will involve four team members, each assigned specific technical responsibilities based on their areas of expertise. To encourage shared leadership and accountability, the team will rotate the Team Lead role monthly. Technical roles may also be adjusted throughout development based on project requirements and workload distribution.

14.1 Team Leadership

Team Lead / Scrum Master (Rotating Monthly)

The Team Lead position will rotate among team members on a monthly basis. The Team Lead will be responsible for coordinating team activities, monitoring project progress, and ensuring deadlines are met.

Key responsibilities include:

- Monitoring project deadlines and ensuring deliverables are submitted on time.
- Assigning and tracking tasks using Jira.
- Acting as the Scrum Master by coordinating sprint planning, progress meetings, and sprint reviews.
- Identifying development blockers and coordinating efforts to resolve them.
- Ensuring team members remain accountable for their assigned responsibilities.
- Facilitating communication and collaboration between team members.

14.2 Technical Roles and Responsibilities

Team Member	Assigned Role	Responsibilities
Erioluwa Olubadejo	Database Engineer	Responsible for designing, implementing, and maintaining the project's database. Ensures data is organized,

		accessible, and properly structured for use by the application. Oversees database-related code changes, integrations, and updates.
Elias Poitras-Whitecalf	Backend Engineer	Responsible for developing and maintaining backend functionality, implementing API endpoints, and supporting integration between the multi-agent system, database, and web application. Ensures backend code is functional, maintainable, and properly tested. Reviews backend-related code changes.
Adol Awan	Backend Engineer	Responsible for developing backend methods and services, supporting API development, and ensuring reliable communication between system components. Collaborates on backend integration, testing, debugging, and reviewing backend-related code changes.
Sam Haque	AI Engineer	Responsible for researching Agentic AI technologies, evaluating AI models, validating AI-generated outputs, and analyzing API usage costs. Oversees architectural considerations related to the multi-agent system and leads AI model selection decisions based on technical analysis and team input.

14.3 RACI Matrix

Deliverable / Activity	Sam	Elias	Adol	Erioluwa
Frontend / user interface	**A/R**	C	I	C
AI model evaluation and selection (8.2)	**A/R**	I	R	C
Professor beta testing and validation (15.2)	**A/R**	R	C	C
Agent architecture and Manager Agent orchestration	R	R	**A/R**	R
Candidate Evaluation Agent	R	C	**A/R**	C
Output validation (schemas, scores, constraints)	C	R	**A/R**	C
Bias and fairness testing (7.4.2)	C	I	**A/R**	R
REST API endpoints and authentication	R	**A/R**	R	R
Deterministic ranking and weighted scoring	C	**A/R**	R	I
Scheduling Agent and schedule validator	C	**A/R**	R	R
Automated testing and CI/CD pipelines	R	**A/R**	R	R
Keyword Extraction Agent	C	R	C	**A/R**
Regex anonymization pipeline	C	C	R	**A/R**
Prompt/SQL injection defenses (7.4.1)	C	C	R	**A/R**
Database integration (read-only access, results storage)	I	R	C	**A/R**
Sponsor communication, documentation, team logbook	*Current (rotating) Team Lead is A; all members are R for their own areas*			

A/R - Accountable and Responsible: owns the outcome, signs off, and does the work

R – Responsible: does the work

C – Consulted: gives input before decisions

I – Informed: kept up to date on progress

14.4 Shared Team Responsibilities

In addition to their primary technical roles, all team members will contribute to the overall development and successful completion of the project.

Shared responsibilities include:

- Providing regular progress updates to the current Team Lead.
- Reporting technical challenges, delays, and development blockers.
- Participating in sprint planning, team meetings, and project evaluations.
- Contributing to testing, debugging, documentation, and code reviews.
- Collaborating on system integration and supporting other team members when necessary.
- Maintaining assigned tasks and progress updates in Jira.
- Participating in major architectural and design decisions.

- Frontend development and user interface implementation will be coordinated among team members during Sprint 2, with responsibilities assigned based on availability and development progress.

Technical responsibilities and leadership assignments may be adjusted as the project progresses to accommodate changing requirements, workloads, and development priorities.

15. Measures of success and validation tests

The success of the OPTIMA system will be evaluated based on the consistency, reliability, and relevance of its candidate evaluation and ranking outputs. Since TA hiring decisions are inherently subjective and can vary depending on the preferences and priorities of individual professors, validation will focus on how closely the system's recommendations align with established hiring criteria and professor evaluations rather than treating any single ranking as an objectively correct result.

15.1 Output Consistency and Reliability

The system should produce consistent results when provided with the same input documents, evaluation criteria, and weighting configurations across multiple runs. Repeated evaluations will be conducted using identical inputs to assess whether the agents generate stable candidate scores, rankings, and supporting explanations. Significant variations between runs will be investigated to identify potential inconsistencies in agent behavior or evaluation logic.

15.2 Professor-Based Beta Testing

Beta testing will involve real professors evaluating a subset of TA candidates for a given position. Professors will independently assess and rank the selected candidates, while also explaining their evaluation process and identifying the factors they consider most important when making hiring decisions.

These evaluation criteria and relative priorities will then be translated into the OPTIMA system through configurable criteria and weightings. The system's resulting scores and rankings will be compared against the professors' assessments to determine how closely its recommendations align with their expectations. This process will help evaluate the relevance of the system's ranking methodology and identify areas where the evaluation criteria, weighting configurations, or agent instructions may require refinement.

15.3 Historical Hiring Comparison

Where historical hiring data is available, the system's rankings will also be compared against previous TA hiring outcomes. This will provide an additional reference point for evaluating whether the system produces recommendations that are reasonably aligned with past hiring decisions.

However, historical hiring outcomes should not be treated as an objective measure of correctness. Previous decisions may have been influenced by individual professor preferences, course-specific requirements, and other subjective considerations that may not be fully represented in the available data. As a result, historical comparisons will serve as a supplementary validation method rather than a definitive measure of system accuracy.

15.4 Limitations of Validation

Ultimately, the measures of success for OPTIMA are limited by the subjective nature of TA hiring. There is no universally correct ranking of candidates, as the suitability of an applicant depends on the priorities and requirements of the professor and the position. Therefore, the system will be evaluated primarily on its ability to produce consistent, explainable, and relevant recommendations that align with the configured evaluation criteria.

OPTIMA is intended to function as a decision-support tool rather than replace professor judgment. Professors will retain final authority over TA selection, and the system's rankings should be interpreted as recommendations to support, rather than determine, hiring decisions.

15.5 Security and Fairness Testing.

Before the final demonstration, the system will be checked against the applicant-privacy specification by inspecting outbound LLM payloads generated from test records containing known identifiers. These tests will be run at the end of each sprint so regressions are caught early. Fairness will be checked using pairs of synthetic applications that are identical except for gender-coded language or names; scores and rankings for each pair should match.

16. Team Logbook

The team will maintain a logbook using a combination of Jira, GitHub, and an informal shared logbook to document development progress, track individual contributions, and record important project activities.

16.1 Jira – Sprint Planning and Task Management

Jira will serve as the primary tool for tracking project progress and managing development tasks. Tasks will be organized into sprints, assigned to individual team members, and updated regularly to reflect their current status. The rotating Team Lead will act as the Scrum Master, overseeing sprint progress, monitoring deadlines, and ensuring tasks are completed as planned. The team's Jira board structure is shown in Fig. A3.

16.2 GitHub – Version Control and Development History

GitHub will be used to maintain a record of code changes and contributions throughout development. Team members will follow established version control practices, including descriptive commit messages, dedicated branches for new features or fixes, and pull requests before merging changes into the main branch. Pull requests will allow team members to review changes, discuss implementation decisions, and maintain code quality. GitHub's commit and pull request history will also provide a record of technical progress and individual contributions.

16.3 Informal Team Logbook – Additional Documentation

An informal shared logbook will be maintained to record minor development activities, meeting notes, technical challenges, and decisions that may not require dedicated Jira tasks or GitHub issues. Team members will update this logbook as needed to ensure important information is documented and accessible to the entire team.

16.4 Logbook Maintenance Procedures

All team members will be responsible for maintaining accurate records of their work. Jira tasks will be updated as development progresses, GitHub commits and pull requests will document

code changes, and the informal logbook will capture additional activities and discussions. The rotating Team Lead will periodically review these records to ensure project progress is properly documented.

Together, these tools will provide a record of project development, support accountability among team members, and allow the team to review progress, identify challenges, and track decisions throughout the project.

17. Appendix

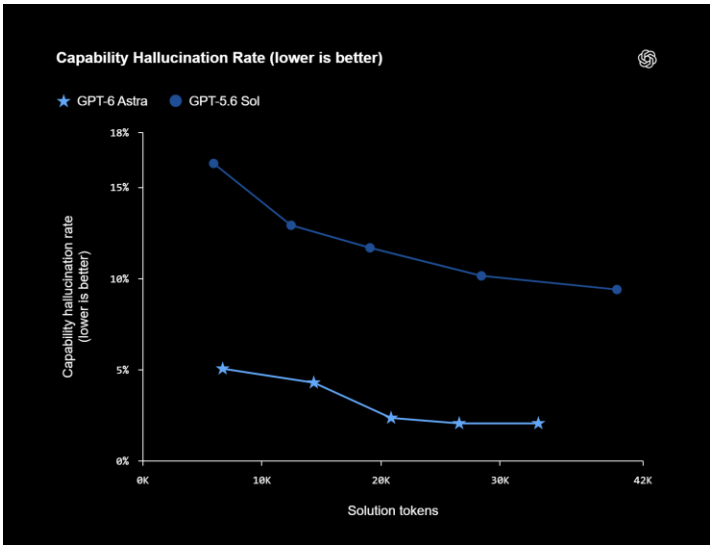


Fig. A1. Capability hallucination rate versus solution tokens for GPT-6 Astra and GPT-5.6 Sol (lower is better). GPT-6 Astra maintains a substantially lower hallucination rate across all token budgets, supporting its consideration for agent reasoning tasks. Reprinted from [38].

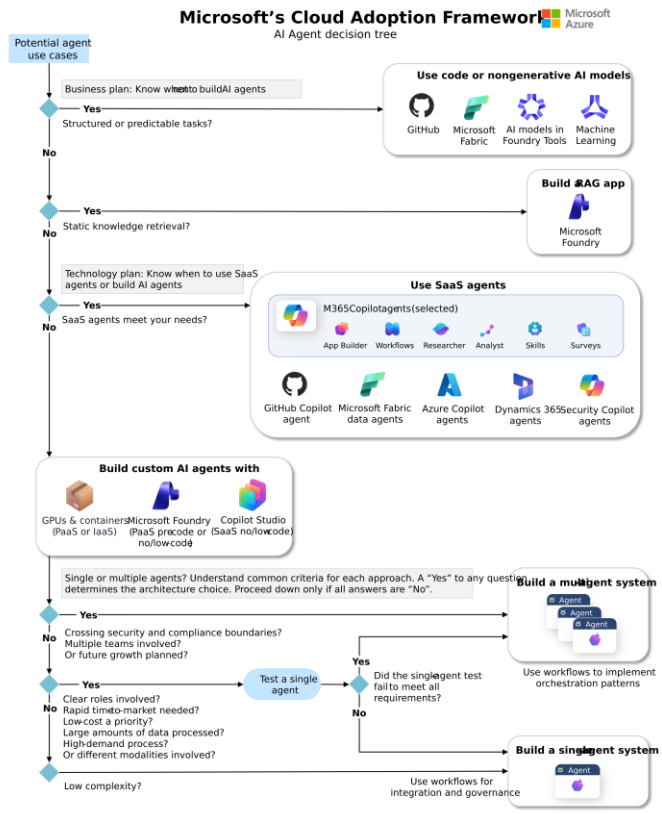


Fig. A2. Microsoft Cloud Adoption Framework AI agent decision tree, used to determine whether a use case calls for non-generative tooling, a RAG application, SaaS agents, or custom agents, and whether a single-agent or multi-agent architecture is appropriate. Reprinted from [42].

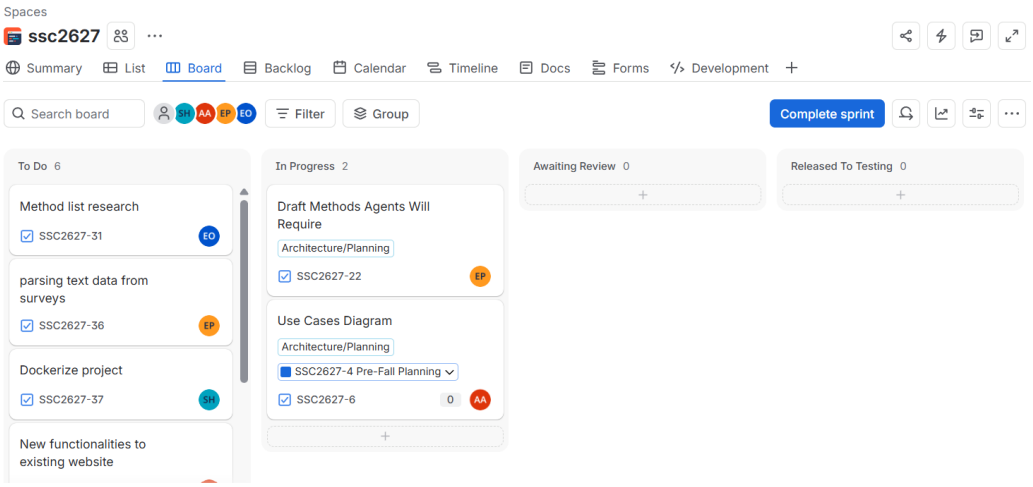


Fig. A3. Team Jira board for project SSC2627, showing sprint tasks organized into To Do, In Progress, Awaiting Review, and Released to Testing columns (see Section 16.1).

Fig. A4: USER STORIES

ID	User story	Acceptance criteria
US-01	As a system administrator, I want to process multiple TA Excel files so that information can be consolidated without a master file.	Process multiple files together; validate required fields and report errors.
US-02	As a system administrator, I want to assign each TA a unique ID so that records can be processed without displaying identities.	Different individuals receive different IDs linked to the correct private identity records.
US-03	As a system administrator, I want to retain the same TA ID across files and future imports so that each person's information remains connected.	Reuse existing mappings; recognized TAs do not receive new IDs.
US-04	As a system administrator, I want to flag ambiguous identity records so that identity conflicts can be resolved before processing continues.	Report missing or conflicting identifiers; do not merge records by name alone.

US-05	As a system administrator, I want to replace identifying information with TA IDs in ranking inputs so that personal details are excluded from AI processing.	Exclude names, emails, UCIDs, and identifying content in other fields from AI input copies; preserve originals.
US-06	As a system administrator, I want to maintain a separate private mapping file so that authorized users can recover identities.	Link IDs to original details; exclude the mapping from AI inputs and ordinary ranking exports.
US-07	As a system administrator, I want to restore identities in final scheduling results so that selected TAs can be identified and contacted.	Restore correct details for recognized IDs; flag unknown IDs.
US-08	As an instructor or coordinator, I want to upload requirements for each course so that candidates are assessed against that course's needs.	Associate files with the correct course and term; flag invalid files.
US-09	As an instructor or coordinator, I want to review imported course requirements so that their accuracy can be confirmed.	Display extracted requirements and source text before applying them.

US-10	As an instructor or coordinator, I want to edit course requirements so that errors and changes can be corrected/incorporated.	Save corrections and identify affected rankings as needing recalculation.
US-11	As an instructor or coordinator, I want to identify mandatory requirements so that ineligible candidates are excluded appropriately.	Apply confirmed mandatory requirements before ranking; explain exclusions and identify missing evidence separately.
US-12	As an instructor or coordinator, I want to assign weights to ranking criteria so that important qualifications contribute more to suitability scores.	Save and apply valid weights; display effective weights.
US-13	As an instructor or coordinator, I want to use all applicable course ranking criteria by default so that rankings can be generated without manual configuration.	Apply and display documented default criteria and weights when no custom settings are chosen.
US-14	As an instructor or coordinator, I want to select specific ranking skills or criteria so that rankings reflect course priorities.	Only selected criteria contribute to scores; mandatory eligibility checks remain active.

US-15	As an instructor or coordinator, I want to compare TA availability across courses so that potential scheduling conflicts can be identified.	Check required times against availability and recorded bookings; display conflicts.
US-16	As an instructor or coordinator, I want to preselect TAs for a course or lab so that preferred candidates can be recorded without ranking for that assignment.	Record TA ID, course/lab, times, hours, and status; perform applicable eligibility checks.
US-17	As an instructor or coordinator, I want to track selection status by TA ID so that selected candidates can be distinguished from remaining candidates.	Record status per TA, course, and term; prevent duplicate selections for the same assignment.
US-18	As an instructor or coordinator, I want to view a TA's selections across courses so that scheduling conflicts and workload can be managed.	Show assignments, booked times, and hours against the same ID; flag conflicts.
US-19	As an instructor or coordinator, I want to update or remove selections so that staffing decisions can change.	Update status, reserved capacity, and remaining vacancies according to the agreed booking policy.

US-20	As an instructor or coordinator, I want to specify how many TAs are needed so that staffing targets and remaining vacancies can be shown.	Save a positive whole-number target per course; count preselections according to the agreed policy.
US-21	As an instructor or coordinator, I want to rank all eligible candidates even when only a limited number is needed so that alternatives remain available.	Show the full ranking and recommendation cutoff; flag ties and insufficient candidates.
US-22	As an instructor or coordinator, I want to review candidates using their assigned IDs so that suitability can be reviewed without identifying details.	Use persistent IDs in candidate lists, scores, and explanations instead of names, emails, or UCIDs.
US-23	As an instructor or coordinator, I want to see why each TA received their rank so that recommendations can be assessed.	Show criteria, supporting information, score contributions, and relevant gaps.
US-24	As an instructor or coordinator, I want to flag incomplete TA information so that missing data is not mistaken for a confirmed lack of qualifications.	Mark missing values clearly and explain their effect on eligibility or ranking.

US-25	As an instructor or coordinator, I want to recalculate rankings when inputs or settings change so that recommendations remain current.	Use the latest confirmed data and settings; identify outdated rankings.
US-26	As an instructor or coordinator, I want to record final candidate selections so that recommendations can become confirmed assignments.	Save assignments after rechecking current availability and workload; flag conflicts before confirmation.
US-27	As an authorized user, I want to export rankings and final selections so that results can be reviewed and shared.	Include course, term, TA ID, rank, score, and status; require separate authorization to restore identities.

18. References

[1] IBM, “What is artificial intelligence (AI)?” Accessed: Nov. 6, 2026. [Online]. Available: https://www.ibm.com/think/topics/artificial-intelligence

[2] IBM, “What are AI agents?” Accessed: Nov. 6, 2026. [Online]. Available: https://www.ibm.com/think/topics/ai-agents

[3] IBM, “What is agentic AI?” Accessed: Nov. 6, 2026. [Online]. Available: https://www.ibm.com/think/topics/agentic-ai

[4] IBM, “What is a multiagent system?” Accessed: Nov. 6, 2026. [Online]. Available: https://www.ibm.com/think/topics/multiagent-system

[5] IBM, “What are large language models (LLMs)?” Accessed: Nov. 6, 2026. [Online]. Available: https://www.ibm.com/think/topics/large-language-models

[6] OpenAI, “OpenAI Agents SDK for Python.” Accessed: Nov. 6, 2026. [Online]. Available: https://openai.github.io/openai-agents-python/

[7] University of Calgary, Department of Philosophy, “Graduate fees and funding.” Accessed: Nov. 7, 2026. [Online]. Available: https://arts.ucalgary.ca/philosophy/future-students/graduate/fees-and-funding

[8] Scikit-learn Developers, “ndcg_score.” Accessed: Nov. 7, 2026. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.ndcg_score.html

[9] Google, “Employee scheduling.” Accessed: Nov. 7, 2026. [Online]. Available: https://developers.google.com/optimization/scheduling/employee_scheduling

[10] IBM, “What is natural language processing (NLP)?” Accessed: Nov. 6, 2026. [Online]. Available: https://www.ibm.com/think/topics/natural-language-processing

[11] N. Selvaraj, “What is retrieval augmented generation (RAG)? A complete guide,” DataCamp, May 28, 2026. Accessed: Nov. 6, 2026. [Online]. Available: https://www.datacamp.com/blog/what-is-retrieval-augmented-generation-rag

[12] IBM, “What is a vector database?” Accessed: Nov. 6, 2026. [Online]. Available: https://www.ibm.com/think/topics/vector-database

[13] IBM, “What are embeddings?” Accessed: Nov. 6, 2026. [Online]. Available: https://www.ibm.com/think/topics/embedding

[14] Google Cloud, “What is semantic search?” Accessed: Nov. 6, 2026. [Online]. Available: https://cloud.google.com/discover/what-is-semantic-search

[15] Cloudflare, “What is HTTP (Hypertext Transfer Protocol)?” Accessed: Nov. 6, 2026. [Online]. Available: https://www.cloudflare.com/learning/ddos/glossary/hypertext-transfer-protocol-http/

[16] Red Hat, “What is a REST API?” Accessed: Nov. 6, 2026. [Online]. Available: https://www.redhat.com/en/topics/api/what-is-a-rest-api

[17] IBM, “What is a relational database?” Accessed: Nov. 6, 2026. [Online]. Available: https://www.ibm.com/think/topics/relational-databases

[18] Oracle, “What is MySQL?” Accessed: Nov. 6, 2026. [Online]. Available: https://www.oracle.com/ca-en/mysql/what-is-mysql/

[19] JSON.org, “Introducing JSON.” Accessed: Nov. 6, 2026. [Online]. Available: https://www.json.org/json-en.html

[20] MotherDuck, “CSV (Comma-Separated Values).” Accessed: Nov. 6, 2026. [Online]. Available: https://motherduck.com/glossary/csv/

[21] Fairlearn Contributors, “Common fairness metrics.” Accessed: Nov. 7, 2026. [Online]. Available: https://fairlearn.org/main/user_guide/assessment/common_fairness_metrics.html

[22] IBM, “What is human-in-the-loop?” Accessed: Nov. 6, 2026. [Online]. Available: https://www.ibm.com/think/topics/human-in-the-loop

[23] National Institute of Standards and Technology, “Pseudonymization,” CSRC Glossary. Accessed: Nov. 6, 2026. [Online]. Available: https://csrc.nist.gov/glossary/term/pseudonymization

[24] IBM, “Structured vs. unstructured data.” Accessed: Nov. 6, 2026. [Online]. Available: https://www.ibm.com/think/topics/structured-vs-unstructured-data

[25] W3Schools, “Pandas Introduction.” Accessed: Nov. 6, 2026. [Online]. Available: https://www.w3schools.com/python/pandas/pandas_intro.asp

[26] OWASP GenAI Security Project, “LLM01:2025 Prompt injection,” OWASP Top 10 for LLM Applications 2025. Accessed: Nov. 8, 2026. [Online]. Available: https://genai.owasp.org/llmrisk/llm01-prompt-injection/

[27] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection,” in Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec ‘23), Copenhagen, Denmark, 2023, pp. 79–90, doi: 10.1145/3605764.3623985.

[28] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, “Formalizing and benchmarking prompt injection attacks and defenses,” in Proc. 33rd USENIX Security Symp., Philadelphia, PA, USA, Aug. 2024, pp. 1831–1847.

[29] OpenAI, “Guardrails,” OpenAI Agents SDK Documentation. Accessed: Nov. 8, 2026. [Online]. Available: https://openai.github.io/openai-agents-python/guardrails/

[30] K. Wilson and A. Caliskan, “Gender, race, and intersectional bias in resume screening via language model retrieval,” in Proc. AAAI/ACM Conf. AI, Ethics, and Society, vol. 7, no. 1, 2024, pp. 1578–1590, doi: 10.1609/aies.v7i1.31748.

[31] J. M. Madera, M. R. Hebl, and R. C. Martin, “Gender and letters of recommendation for academia: Agentic and communal differences,” J. Appl. Psychol., vol. 94, no. 6, pp. 1591–1599, 2009, doi: 10.1037/a0016539.

[32] R. Schwartz, A. Vassilev, K. Greene, L. Perine, A. Burt, and P. Hall, “Towards a standard for identifying and managing bias in artificial intelligence,” NIST, Gaithersburg, MD, USA, NIST SP 1270, Mar. 2022, doi: 10.6028/NIST.SP.1270.

[33] P. Wang et al., “Large language models are not fair evaluators,” in Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (ACL), Bangkok, Thailand, Aug. 2024, pp. 9440–9450, doi: 10.18653/v1/2024.acl-long.511.

[34] S. Mohammed, L. Budach, M. Feuerpfeil, N. Ihde, A. Nathansen, N. Noack, H. Patzlaff, F. Naumann, and H. Harmouch, “The effects of data quality on machine learning performance on tabular data,” Information Systems, vol. 132, Art. no. 102549, Jul. 2025, doi: 10.1016/j.is.2025.102549.

[35] Scikit-learn Developers, “Model selection and evaluation,” Scikit-learn. Accessed: Nov. 5, 2026. [Online]. Available: https://scikit-learn.org/stable/model_selection.html

[36] Scikit-learn Developers, “Preprocessing data,” Scikit-learn. Accessed: Nov. 5, 2026. [Online]. Available: https://scikit-learn.org/stable/modules/preprocessing.html

[37] S. V, “Agentic AI and machine learning: Key distinctions,” Softobiz, Jul. 2026. Accessed: Nov. 6, 2026. [Online]. Available: https://softobiz.com/insights/agentic-ai-vs-machine-learning/

[38] OpenAI, “GPT-6 Astra: A new generation of intelligence,” OpenAI, 2026. Accessed: Nov. 6, 2026. [Online]. Available: https://openai.com/index/gpt-6-astra/

[39] Anthropic, “Models overview,” Claude Platform Docs. Accessed: Nov. 6, 2026. [Online]. Available: https://platform.claude.com/docs/en/models/overview

[40] Google DeepMind, “Gemini 4 Argon,” Google DeepMind, Oct. 13, 2026. Accessed: Nov. 6, 2026. [Online]. Available: https://deepmind.google/models/model-cards/gemini-4-argon/

[41] LangChain, “LangChain: The open agent platform to own your intelligence,” LangChain. Accessed: Nov. 6, 2026. [Online]. Available: https://www.langchain.com/

[42] Microsoft, “Single agent or multiple agents,” Microsoft Learn, Dec. 10, 2025. Accessed: Nov. 6, 2026. [Online]. Available: https://learn.microsoft.com/en-us/azure/cloud-adoption-framework/ai-agents/single-agent-multiple-agents

[43] Ashby, “What an ATS should be,” Ashby. Accessed: Nov. 6, 2026. [Online]. Available: https://www.ashbyhq.com/